TOTALISATION

Réconciliation avec le cycle de vie de la réalisation du logiciel

Software Development Life Cycle (SDLC)

PARTIE A

AUTEUR

Léo Michel Bélanger

IMAGES ET COUVERTURE

Soléane Bélanger

PUBLIÉ PAR ORIGINALIUM | TTLSTT~

Montréal, Québec, Canada | 2026

Série ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés

Images et couverture par Soléane Bélanger

Publié par ORIGINALIUM | ŢŢĻṢŢ~

Aucune partie de cet ouvrage ne peut être reproduite, transmise ou distribuée sous quelque forme ou par quelque moyen que ce soit, électronique ou mécanique, y compris la photocopie, l'enregistrement ou tout autre système de stockage et de récupération de l'information, sans l'autorisation écrite préalable de l'auteur.

Première édition, 2026

Imprimé au Canada

Table des matières

Préambule - Le SDLC : une promesse souvent trahie

Le paradoxe fondamental

Pourquoi la réconciliation est une nécessité

Ce que Totalisation apporte de nouveau

Chapitre 1 - Qu'est-ce que le SDLC vu par Totalisation ?

La définition traditionnelle

Les grandes familles : Waterfall, Agile, DevOps

Ce que chaque approche a vu juste - et ce qu'elle a manqué

La vision ŢŢĻṢŢ~ : le flux continu

Chapitre 2 - La fragmentation du SDLC : une blessure à guérir

Pourquoi on a découpé le cycle en phases séparées

Les coûts cachés de la fragmentation

La perte de sens quand on ne voit que sa phase

Le syndrome du "ce n'est pas mon problème"

Comment Totalisation diagnostique cette fracture

Chapitre 3 - Le Waterfall et sa réconciliation avec Totalisation

La noblesse de l'intention originale

Les forces réelles du Waterfall

Les faiblesses que Totalisation reconnaît sans condamner

Intégrer la sagesse du Waterfall dans une vision totalisante

Le Waterfall n'est pas mort : il est mal utilisé

Chapitre 4 - Agile et la réconciliation avec Totalisation

Le manifeste Agile : une révolution nécessaire

Ce qu'Agile a réparé - et ce qu'il a cassé en retour

L'agilité mal comprise

La vision ŢŢĻŞŢ~ sur l'Agile

Agile et Totalisation : une réconciliation productive

Note de fin de Partie A - Aperçu de la Partie B

TOTALISATION - ŢŢĻŞŢ~

DIMENSION 1 - SUBSTANTIF LINGUISTIQUE

« Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser. »

DIMENSION 2 - PHILOSOPHIE ORIGINALIUM

« Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. »

La Méthode BEASTEST - Niveaux de maturité

La méthode BEASTEST structure la progression de la maturité en sept niveaux, chacun représentant un palier d'évolution dans la compréhension et la mise en oeuvre du concept de Totalisation :

Niveau

Couleur

Signification dans le contexte de la Totalisation

1

ROUGE

Prise de conscience initiale - on reconnaît qu'un problème existe

2

ORANGE

Compréhension des fragments - on commence à voir les pièces séparées

3

JAUNE

Liaison des éléments - on perçoit les connexions entre les phases

4

VERT

Intégration active - on agit en tenant compte de l'ensemble

5

BLEU

Fluidité systémique - le flux continu devient naturel

6

INDIGO

Profondeur philosophique - on vit la Totalisation comme une posture

7

VIOLET

Totalisation accomplie - avant, pendant et après, en continu, sans interruption

PRÉAMBULE

Le SDLC : une promesse souvent trahie

Le paradoxe fondamental

Il y a quelque chose de profondément troublant dans le paysage actuel du développement logiciel. Quelque chose que tu as peut-être déjà ressenti, que tu as peut-être mis des mots dessus ou gardé pour toi, sans savoir tout à fait comment l'exprimer. C'est une sensation d'inconfort face à une contradiction qui se répète sans relâche, de projet en projet, d'organisation en organisation,

d'équipe en équipe. La voici dans toute sa brutalité : on ne cesse jamais de mieux définir, mieux structurer, mieux cadrer le cycle de vie de la réalisation du logiciel - et pourtant, les projets continuent d'échouer, de dérailler, de livrer moins que ce qu'on avait promis.

Ce paradoxe n'est pas une anomalie. Ce n'est pas le résultat d'une mauvaise chance, d'une équipe incompétente ou d'un client difficile. C'est le symptôme systémique d'une fracture bien plus profonde : la fracture entre la façon dont on conçoit le cycle de vie logiciel et la façon dont la réalité d'un projet se déroule effectivement. On a beau adopter le Waterfall, l'Agile, le DevOps ou une kyrielle d'autres méthodologies - Scrum, Kanban, SAFe, LeSS, DAD - le problème fondamental demeure intact : on traite un flux naturel et continu comme s'il était une séquence de boîtes étanches à cocher.

Et c'est précisément là que ŢŢĻŞŢ~ entre en scène. Non pas pour remplacer ces méthodologies. Non pas pour en proposer une nouvelle, encore plus complexe, encore plus chargée de cérémonies et de rituels. Mais pour opérer quelque chose de plus fondamental : une réconciliation. Une réconciliation entre toi et le cycle de vie. Entre ton équipe et la réalité du projet. Entre la vision initiale et le livrable final. Entre l'avant, le pendant et l'après.

> *« La vraie question n'est pas : "Quelle méthodologie devrions-nous adopter ?" La vraie question est : "Pourquoi nos méthodologies ne guérissent-elles pas la fracture ?" »*
>
> - Léo Michel Bélanger, Philosophie ORIGINALIUM

Pourquoi la réconciliation est une nécessité

Le mot "réconciliation" est intentionnel. On ne réconcilie pas deux choses qui se sont toujours bien entendues. On réconcilie deux réalités qui se sont éloignées l'une de l'autre, qui ont développé des tensions, des malentendus, peut-être même une certaine hostilité. Et c'est exactement ce qui s'est passé entre les équipes de développement logiciel et le concept même du SDLC.

Le SDLC, dans son intention originale, était une promesse magnifique. C'était la promesse qu'on pouvait maîtriser la complexité de créer quelque chose d'aussi abstrait et d'aussi évolutif qu'un logiciel. C'était la promesse d'un chemin balisé, d'une route qu'on pouvait parcourir avec confiance depuis l'idée jusqu'au déploiement. Mais quelque chose s'est brisé entre la promesse et la réalité. Ce quelque chose, c'est la fragmentation.

Quand on a commencé à découper le cycle de vie en phases distinctes, chacune avec ses rôles, ses livrables, ses critères d'entrée et de sortie, on a sans doute cru bien faire. On pensait apporter de la clarté là où il y avait du chaos. Mais ce qu'on a réellement produit, c'est une division artificielle d'un processus qui, par nature, est organique et continu. On a transformé un fleuve en une série de bassins séparés. Et quand l'eau ne coule plus librement, elle stagne. Elle s'acidifie. Elle perd sa vitalité.

La réconciliation que propose ŢŢŢŞŢ~ n'est donc pas un luxe intellectuel. C'est une nécessité pratique, urgente, que chaque organisation qui construit des logiciels doit embrasser si elle veut cesser de perpétuer les mêmes erreurs coûteuses, les mêmes désillusions collectives, les mêmes projets qui finissent soit dans la médiocrité, soit dans l'abandon pur et simple.

Ce que Totalisation apporte de nouveau à la compréhension du SDLC

Totalisation n'est pas une méthode de gestion de projet. Ce point doit être absolument clair dès le départ, parce que la tentation sera forte de la ranger dans cette catégorie, de lui attribuer des cérémonies, des rôles, des artefacts. Résiste à cette tentation. Totalisation est une posture philosophique avant d'être quoi que ce soit d'autre.

Ce que Totalisation apporte de fondamentalement nouveau, c'est la conscience de la continuité. C'est la compréhension que le cycle de vie d'un logiciel n'est jamais une ligne droite que l'on parcourt d'un point A à un point Z. C'est un flux vivant, dans lequel chaque moment contient en lui l'écho de ce qui l'a précédé et le germe de ce qui le suivra. Quand tu es dans la phase de développement, tu portes déjà en toi les décisions d'architecture prises lors de la conception - et tu portes déjà la responsabilité envers les équipes qui vont devoir maintenir ce que tu construis.

Cette conscience de la continuité se décline en deux dimensions, que nous avons déjà posées en ouverture de cet ouvrage. La première est linguistique et conceptuelle : totaliser, c'est additionner, c'est synthétiser, c'est considérer l'ensemble. La seconde est philosophique, propre à ORIGINALIUM : totaliser, c'est rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. Ces deux dimensions ne se contredisent pas. Elles se complètent. Ensemble, elles dessinent une vision du SDLC qui transcende les limitations de toutes les approches existantes, non pas en les effaçant, mais en les englobant dans quelque chose de plus vaste.

Point de philosophie fondamental

Totalisation ne te demande pas d'abandonner ta méthodologie actuelle. Elle te demande de changer le regard que tu portes sur elle. La même réalité, vue différemment, produit des résultats radicalement différents. C'est l'essence de cette réconciliation.

CHAPITRE 1

Qu'est-ce que le SDLC vu par Totalisation ?

La définition traditionnelle du Software Development Life Cycle

Le Software Development Life Cycle - que l'on traduit en français par "cycle de vie de la réalisation du logiciel" - est, dans sa définition la plus académique, un processus structuré qui décrit les étapes par lesquelles un système logiciel passe, depuis sa conception initiale jusqu'à son retrait définitif. Cette définition, aussi froide et technique qu'elle puisse paraître, contient pourtant une vérité fondamentale que l'on tend à oublier dès qu'on commence à l'opérationnaliser : le mot "vie".

Le SDLC n'est pas un processus de production industrielle. Ce n'est pas une chaîne de montage où des pièces inertes entrent à une extrémité et sortent assemblées à l'autre. C'est un cycle de vie. Et un cycle de vie implique quelque chose

d'organique, de dynamique, de vivant. Il implique des allers-retours, des adaptations, des apprentissages, des moments de croissance et des moments de remise en question. Il implique surtout une continuité fondamentale que les définitions traditionnelles ont tendance à trahir en découpant ce cycle en phases distinctes et séquentielles.

Historiquement, le SDLC a été formalisé dans les années 1960 et 1970, à une époque où le développement logiciel cherchait à s'inspirer des disciplines d'ingénierie traditionnelles - le génie civil, le génie mécanique - pour gagner en rigueur et en prévisibilité. Cette inspiration était compréhensible, peut-être même inévitable à ce stade de maturité de la discipline. Mais elle a introduit une hypothèse fondamentalement incorrecte : l'hypothèse que construire un logiciel ressemble à construire un pont ou une automobile, c'est-à-dire que les exigences peuvent être complètement définies avant que la construction commence, et que les phases peuvent se succéder sans chevauchement significatif.

Les grandes familles : Waterfall, Agile, DevOps et leurs variantes

Pour comprendre ce que Totalisation apporte, il faut d'abord avoir une vision claire des grandes familles de méthodologies SDLC qui ont structuré notre pensée collective au cours des dernières décennies. Non pas pour les juger, mais pour les comprendre dans leur intention profonde.

Le **modèle en cascade (Waterfall)**, formalisé par Winston W. Royce en 1970, est souvent présenté comme le point de départ de toutes les discussions sur le SDLC. Il propose une progression linéaire et séquentielle à travers des phases bien définies : les exigences, la conception, l'implémentation, la vérification et la maintenance. Chaque phase doit être complète avant que la suivante commence. C'est une vision qui cherche la prévisibilité, la documentation et la traçabilité. Sa

noblesse réside dans son désir d'ordre face au chaos. Ses limites résident dans sa rigidité face au changement.

Le mouvement **Agile**, formalisé par le Manifeste Agile de 2001, est né précisément en réaction aux limitations du Waterfall. Il propose une approche itérative et incrémentale, centrée sur la collaboration, l'adaptation au changement et la livraison continue de valeur. Scrum, Kanban, Extreme Programming (XP) sont autant de cadres qui s'inscrivent dans cette philosophie. L'Agile a apporté une souplesse vitale au SDLC. Mais dans son déploiement réel, il a souvent produit ses propres dérives : la vitesse sans profondeur, l'itération sans apprentissage cumulatif, la flexibilité sans vision à long terme.

Le mouvement **DevOps**, qui s'est imposé dans les années 2010, a cherché à réconcilier le développement et les opérations, deux mondes qui s'étaient développés en silos quasi hermétiques. DevOps promeut l'automatisation, la livraison continue (CI/CD), la surveillance et une culture de collaboration entre les équipes de développement et d'exploitation. C'est une avancée réelle, mais qui reste souvent limitée à une dimension technique et organisationnelle, sans aborder les questions philosophiques de fond sur la nature même du cycle de vie.

Au-delà de ces trois grandes familles, on trouve une constellation de variantes et d'hybrides : le modèle en spirale de Boehm, le Rational Unified Process (RUP), le modèle en V, le Scaled Agile Framework (SAFe), le Large-Scale Scrum (LeSS), et bien d'autres encore. Chacun de ces cadres représente une tentative de répondre à des limitations spécifiques identifiées dans les approches précédentes. Chacun apporte quelque chose de réel. Et chacun, à sa façon, perpétue une vision fondamentalement fragmentée du cycle de vie.

Ce que chaque approche a vu juste - et ce qu'elle a manqué

ŢŢĻŞŢ~ ne juge pas. Totalisation réconcilie. Et pour réconcilier, il faut d'abord reconnaître les forces de chaque approche avec autant de générosité qu'on reconnaît leurs limites. Voici donc, pour chacune des grandes familles, un regard totalisateur honnête et équitable.

Approche	Ce qu'elle a vu juste	Ce qu'elle a manqué
Waterfall	La nécessité de la documentation, de la traçabilité et de la planification rigoureuse. L'importance de comprendre complètement ce qu'on construit avant de le construire.	La nature fondamentalement évolutive des exigences. Le fait que la connaissance se construit pendant la réalisation, pas seulement avant.
Agile	La centralité du changement et de l'adaptation. La valeur de la livraison fréquente et de la rétroaction rapide. La primauté des individus sur les processus.	La nécessité d'une vision architecturale cohérente sur le long terme. Le risque de la dette technique accumulée sprint après sprint. L'apprentissage cumulatif d'une itération à l'autre.
DevOps	L'élimination des silos entre développement et opérations. L'automatisation comme levier de qualité et de vitesse. La surveillance continue en production.	La dimension humaine et culturelle profonde du changement. La question de sens pour les équipes. La vision totalisante qui dépasse la seule dimension technique.

La vision ŢŢĻŞŢ~ : le SDLC comme flux continu, pas comme séquence de phases

Voici le coeur de ce premier chapitre, et peut-être le principe le plus important de tout cet ouvrage. Totalisation propose une vision radicalement différente du SDLC

: non pas comme une séquence de phases à traverser, mais comme un flux continu à habiter.

Qu'est-ce que cela signifie concrètement ? Cela signifie que tu n'es jamais "dans" la phase d'analyse et "hors" de la phase de conception. Tu n'es jamais "dans" le développement et "hors" des tests. À tout moment du cycle de vie, tu es simultanément en train d'analyser ce que tu fais, de concevoir la suite, d'implémenter le présent, de tester ce qui existe et d'anticiper la maintenance de ce que tu livres. Ces dimensions coexistent. Elles ne se succèdent pas dans le temps - elles se superposent dans l'espace de ta conscience professionnelle.

> *« Le SDLC n'est pas un escalier que tu montes une marche à la fois. C'est un fleuve dans lequel tu es déjà immergé. La question n'est pas de savoir quelle marche tu es en train de franchir. La question est : est-ce que tu nages avec le courant ou contre lui ? »*
>
> - ṰṰḶṢṬ~ - Philosophie ORIGINALIUM

Cette vision du flux continu n'est pas une abstraction poétique sans conséquences pratiques. Elle transforme profondément la façon dont tu prends des décisions, dont tu communiques avec ton équipe, dont tu interagis avec tes parties prenantes. Elle te libère de la prison mentale des phases étanches. Elle te permet de voir comment une décision prise aujourd'hui dans ton code va résonner demain dans la maintenance, dans six mois dans l'évolution des fonctionnalités, dans trois ans dans la dette technique que ton équipe aura à porter.

C'est cette vision panoramique, cette capacité à embrasser simultanément l'avant, le pendant et l'après, qui constitue l'essence de Totalisation appliquée au SDLC. Et c'est pour t'emmener vers cette vision que cet ouvrage a été écrit.

CHAPITRE 2

La fragmentation du SDLC : une blessure à guérir

Pourquoi on a découpé le cycle en phases séparées

La fragmentation du SDLC n'est pas née d'une mauvaise intention. C'est essentiel de le comprendre pour pouvoir s'en libérer. Si on a découpé le cycle de vie en phases distinctes, c'est parce que face à la complexité croissante des projets logiciels, face à des équipes de plus en plus grandes, face à des organisations de plus en plus structurées, le découpage semblait être la seule façon raisonnable de maintenir un semblant de contrôle et de coordination.

Le découpage par phases répondait à des besoins réels. Il permettait d'assigner des responsabilités claires à des individus ou des équipes spécifiques. Il créait des points de validation formels où les parties prenantes pouvaient vérifier que le projet progressait dans la bonne direction avant d'autoriser la suite. Il offrait un vocabulaire partagé - "on est en phase de conception", "on vient d'entrer en phase de test" - qui facilitait la communication au sein d'organisations complexes.

En ce sens, la fragmentation a été une réponse adaptée à un contexte spécifique. Le problème, c'est que cette réponse contextuelle est devenue une vérité absolue, une façon de voir le monde logiciel qui s'est cristallisée en dogme. Et comme tous les dogmes, elle a commencé à produire des effets pervers bien au-delà du contexte original pour lequel elle avait été conçue.

Les coûts cachés de la fragmentation : erreurs en cascade, silos humains

Le premier coût caché de la fragmentation, c'est ce qu'on pourrait appeler les erreurs en cascade. Quand les phases sont étanches, les erreurs commises dans une phase ne sont découvertes que plusieurs phases plus tard - parfois beaucoup plus tard. Une exigence mal comprise lors de la phase d'analyse devient une fonctionnalité mal conçue lors de la phase de conception, qui devient une implémentation incorrecte lors du développement, qui devient un bug découvert lors des tests, qui nécessite un retour en arrière vers le code, puis vers la conception, puis parfois vers les exigences elles-mêmes. Le coût de correction d'une erreur croît de façon exponentielle plus on la détecte tardivement dans le cycle de vie. Ce n'est pas une théorie - c'est une réalité que de nombreuses études sur la qualité logicielle ont confirmée.

Mais le deuxième coût caché est encore plus insidieux, parce qu'il est invisible dans les tableaux de bord de projet : les silos humains. Quand tu organises ton cycle de vie en phases distinctes, tu crées inévitablement des silos humains correspondants. Les analystes d'affaires habitent la phase d'analyse. Les architectes habitent la phase de conception. Les développeurs habitent la phase de développement. Les testeurs habitent la phase de test. Et les équipes d'exploitation habitent la phase de déploiement et de maintenance.

Ces silos ne sont pas seulement organisationnels. Ils sont mentaux, culturels, émotionnels. Ils créent des identités professionnelles étroitement liées à une phase spécifique du cycle, et par conséquent, une difficulté profonde à s'identifier à l'ensemble du projet, à se sentir responsable de ce qui se passe dans les autres phases. Et c'est là que naît le syndrome que nous allons examiner plus loin : "ce n'est pas mon problème".

La perte de sens quand on ne voit que sa phase

Il y a quelque chose de profondément déshumanisant dans le fait de ne voir que sa phase du SDLC. Imagine un musicien qui ne connaîtrait que sa propre partition, sans jamais entendre la symphonie dans son ensemble. Il pourrait jouer sa partie à la perfection, techniquement impeccable, rigoureusement dans le rythme - et pourtant rester totalement déconnecté du sens de l'oeuvre qu'il est en train d'interpréter.

C'est exactement ce qui arrive à beaucoup de professionnels du logiciel dans des organisations fortement fragmentées. Le développeur qui code une fonctionnalité sans comprendre le problème d'affaires qu'elle résout perd le fil de sens qui donne de la profondeur à son travail. Le testeur qui valide des cas de test sans comprendre l'architecture technique du système qu'il teste se prive de la perspicacité qui lui permettrait de trouver les bugs les plus subtils. L'analyste d'affaires qui rédige des exigences sans avoir conscience des contraintes techniques dans lesquelles elles devront être implémentées produit des spécifications qui vont générer de la frustration à toutes les étapes suivantes.

Cette perte de sens n'est pas anodine. Elle a des conséquences directes sur la qualité du travail produit, sur l'engagement des équipes, sur leur capacité à innover et à résoudre des problèmes complexes. Quand tu ne vois pas la totalité, tu ne peux pas produire la totalité. Tu produis un fragment. Et une collection de fragments, même chacun parfaitement exécuté, ne constitue pas nécessairement un tout cohérent.

Le syndrome du "ce n'est pas mon problème" dans les équipes fragmentées

Le syndrome du "ce n'est pas mon problème" est la manifestation la plus visible et la plus douloureuse de la fragmentation du SDLC. Tu l'as probablement rencontré. Peut-être l'as-tu même expérimenté toi-même, sans t'en rendre compte, parce

que dans un contexte de fragmentation, cette réaction semble parfaitement rationnelle.

Le développeur qui dit "les testeurs vont trouver ça" quand il hésite à corriger un bug potentiel. L'analyste qui dit "c'est aux développeurs de se débrouiller avec les exigences ambiguës". L'équipe de production qui dit "nous n'avons pas été impliqués dans la conception de l'architecture, donc nous ne pouvons pas être tenus responsables des problèmes de performance". Ces réactions ne viennent pas d'un manque d'éthique professionnelle. Elles viennent d'un système qui a organisé les responsabilités de façon si étroite, si cloisonnée, que chacun a appris à défendre son territoire en laissant les "zones grises" entre les phases comme des zones de non-responsabilité collective.

> *« Dans un système fragmenté, personne n'est responsable des espaces entre les phases. Et c'est précisément dans ces espaces que la majorité des problèmes naissent, grandissent et deviennent des crises. »*
>
> - ŢŢĻṢŢ~ - Philosophie ORIGINALIUM

Totalisation nomme ces zones de non-responsabilité collective les "espaces de fragmentation". Ce sont des espaces entre le "ce n'est plus mon problème" et le "ce n'est pas encore mon problème". Des espaces où les exigences ambiguës deviennent des fonctionnalités mal comprises. Où les décisions d'architecture mal documentées deviennent des contraintes de maintenance imprévues. Où les bugs découverts en production ne trouvent pas de propriétaire clair pour les résoudre.

Comment Totalisation diagnostique cette fracture

Le diagnostic que pose Totalisation sur la fracture du SDLC est à la fois simple dans son énoncé et profond dans ses implications. La fracture n'est pas une question de mauvais processus. Ce n'est pas une question de mauvaises personnes. C'est une question de mauvaise ontologie - c'est-à-dire une vision fondamentalement incorrecte de ce que le SDLC est réellement.

Quand tu traites le SDLC comme une séquence de phases distinctes, tu imposes à une réalité continue et organique une structure artificielle et rigide. C'est comme essayer de comprendre le cours d'un fleuve en l'étudiant une tranche de temps à la fois, sans jamais observer comment ces tranches sont reliées. Tu peux accumuler énormément d'informations sur chaque tranche, mais tu ne comprendras jamais le fleuve.

Totalisation propose donc de commencer par guérir cette fracture ontologique avant même de s'attaquer aux processus, aux outils ou aux structures organisationnelles. Ce qui signifie changer fondamentalement la façon dont toi, ton équipe et ton organisation comprenez ce qu'est le SDLC. Le voir non plus comme une série de phases, mais comme un flux continu de réalisation. Non plus comme un processus qu'on exécute, mais comme un espace qu'on habite. Non plus comme une contrainte à gérer, mais comme une réalité à embrasser totalement - avant, pendant et après, en continu, sans interruption.

Totalisation en action - La maturité BEASTEST face à la fragmentation

Un professionnel au niveau ROUGE (niveau 1) ne voit pas la fragmentation comme un problème - il l'a internalisée comme la norme. Au niveau VERT (niveau 4), il commence à agir en tenant compte de l'ensemble du cycle, au-delà de sa phase immédiate. C'est la progression vers VIOLET (niveau 7) qui

représente la pleine incarnation de Totalisation : la conscience constante et fluide de l'avant, du pendant et de l'après dans chacune de ses actions quotidiennes.

CHAPITRE 3

Le Waterfall et sa réconciliation avec Totalisation

Ce que le modèle en cascade a voulu accomplir : la noblesse de l'intention

Avant de parler des limites du Waterfall - et il y a beaucoup à en dire - Totalisation exige qu'on commence par honorer ce que ce modèle a voulu accomplir. Parce que le Waterfall est souvent présenté aujourd'hui comme le symbole de tout ce qui ne va pas dans le développement logiciel, comme le repoussoir parfait contre lequel toutes les approches modernes se définissent. Cette caricature est non seulement injuste - elle est aussi contre-productive, parce qu'elle nous empêche d'apprendre de ce que le Waterfall a compris avec justesse.

Winston W. Royce, dans son article fondateur de 1970 "Managing the Development of Large Software Systems", cherchait à répondre à un problème réel et urgent : comment construire des systèmes logiciels de grande envergure -

pour la NASA, pour le Département de la Défense américain - avec des équipes de centaines de personnes, des budgets colossaux et des enjeux critiques, sans que tout parte dans le chaos ? Sa réponse était modelée sur les disciplines d'ingénierie qu'il connaissait et qui avaient fait leurs preuves pour construire des choses complexes : planifier complètement avant de construire, documenter rigoureusement, vérifier chaque étape avant de passer à la suivante.

L'intention était noble. C'était la conviction qu'on pouvait appliquer à la réalisation du logiciel la même rigueur intellectuelle qu'on appliquait à la construction d'un satellite ou d'un pont. Cette conviction était à la fois la force et la faiblesse du modèle en cascade - et comprendre cela, c'est déjà faire un premier pas vers la réconciliation.

Les forces réelles du Waterfall : documentation, traçabilité, prévisibilité

Les forces du Waterfall ne sont pas des accidents. Elles découlent directement de ses principes fondamentaux et elles restent pertinentes dans des contextes bien spécifiques.

La première force est la **documentation exhaustive**. Le Waterfall exige qu'on documente complètement les exigences avant de commencer à concevoir, qu'on documente complètement la conception avant de commencer à coder. Cette discipline de documentation, que l'Agile a souvent abandonné dans ses excès d'enthousiasme pour la "logique fonctionnelle sur la documentation exhaustive", a une valeur réelle dans des contextes où la continuité à long terme est essentielle. Un système qui doit être maintenu pendant vingt ans par des équipes successives a besoin d'une documentation solide. Un système critique de santé ou de sécurité a besoin d'une traçabilité parfaite entre les exigences et l'implémentation.

La deuxième force est la **traçabilité**. Dans le Waterfall, chaque fonctionnalité peut être tracée depuis l'exigence initiale jusqu'au test qui la valide. Cette traçabilité n'est pas une bureaucratie gratuite. C'est un filet de sécurité qui permet, quand un problème survient, de remonter rapidement à sa source et de comprendre exactement pourquoi une décision a été prise à quel moment du cycle de vie.

La troisième force est la **prévisibilité budgétaire et temporelle**. Dans des contextes contractuels - un gouvernement qui signe un contrat à prix fixe avec un fournisseur logiciel, par exemple - la capacité à estimer avec précision le coût et la durée d'un projet est une nécessité absolue. Le Waterfall, avec sa planification exhaustive en amont, offre cette prévisibilité. Elle est imparfaite, certes - toute prévision l'est - mais elle offre une base contractuelle acceptable dans des contextes où l'Agile, avec son principe de "livraison continue de valeur en s'adaptant au changement", serait difficile à contractualiser.

Les faiblesses que Totalisation reconnaît sans condamner

Totalisation reconnaît les faiblesses du Waterfall avec la même honnêteté qu'elle reconnaît ses forces. La faiblesse centrale du modèle en cascade est son hypothèse implicite de stabilité des exigences. Il suppose que tu peux complètement comprendre et spécifier ce que le système doit faire avant que quiconque ne commence à le construire. Cette hypothèse est fondamentalement incorrecte dans la grande majorité des projets logiciels modernes.

La réalité, que des décennies de pratique du développement logiciel ont prouvée sans ambiguïté, c'est que les parties prenantes ne savent pas vraiment ce qu'elles veulent avant d'avoir vu quelque chose fonctionner. Les exigences évoluent non pas parce que les gens sont inconstants ou incompétents, mais parce que la connaissance se construit pendant la réalisation. Chaque démo, chaque

prototype, chaque version partielle du système révèle des besoins qui n'étaient pas visibles depuis l'abstraction d'une spécification écrite.

La seconde faiblesse majeure est le délai de rétroaction. Dans un modèle purement séquentiel, la rétroaction des utilisateurs n'arrive que très tard dans le cycle, au moment de la livraison finale ou, dans le meilleur des cas, lors d'une phase de test utilisateur en fin de projet. Cela signifie que si une direction fondamentale est mauvaise, on ne le découvre qu'après avoir investi des mois ou des années de travail dans cette direction erronée. Le coût de correction est alors astronomique - parfois suffisant pour faire échouer le projet entier.

Comment intégrer la sagesse du Waterfall dans une vision totalisante

La réconciliation que propose Totalisation avec le Waterfall ne consiste pas à le ressusciter tel quel dans des contextes où il ne fonctionne pas. Elle consiste à extraire sa sagesse profonde et à l'intégrer dans une vision plus large et plus fluide du SDLC.

Cette sagesse profonde, la voici : *comprendre complètement avant d'agir, documenter pour ceux qui viennent après toi, tracer chaque décision pour qu'elle soit compréhensible à travers le temps*. Ces principes ne sont pas propres au Waterfall - ils sont universellement valides dans la réalisation de systèmes complexes. Ce qui était propre au Waterfall, c'était la façon dont il les opérationnalisait : de façon séquentielle, exhaustive et rigide. Totalisation propose de conserver ces principes tout en les libérant de la rigidité séquentielle.

Dans une approche totalisante, tu documentes de façon continue, pas seulement lors d'une phase dédiée à la documentation. Tu traces tes décisions au moment où tu les prends, pas lors d'une revue formelle en fin de phase. Tu cherches à comprendre le problème dans son ensemble à chaque étape du cycle de vie, pas

seulement lors de la phase d'analyse. La sagesse du Waterfall est préservée. Sa prison, le séquentiel rigide, est transcendée.

Le Waterfall n'est pas mort : il est mal utilisé

Il est temps de dire quelque chose que peu de voix dans le monde du développement logiciel modernes osent dire : le Waterfall n'est pas mort, et dans certains contextes, il reste non seulement viable mais préférable à l'Agile. Ce n'est pas une position nostalgique. C'est une position totalisante.

Dans des projets à exigences stables et bien comprises, comme les systèmes embarqués critiques, les systèmes médicaux de classe III, certains systèmes de défense, le Waterfall avec ses phases bien documentées et sa traçabilité rigoureuse est souvent la meilleure approche disponible. Dans ces contextes, "l'adaptation au changement" n'est pas une valeur - la conformité exacte aux exigences certifiées est la valeur. Et le Waterfall excelle à produire cette conformité.

Ce qui est mort, ou du moins ce qui devrait l'être, c'est l'usage aveugle du Waterfall dans des contextes où il ne s'applique pas. C'est l'application mécanique d'un modèle séquentiel à des projets à exigences évolutives, à des marchés changeants, à des utilisateurs dont les besoins se révèlent progressivement. Cette application inadaptée est ce que Totalisation diagnostique comme le vrai problème - pas le Waterfall lui-même.

> *« L'erreur n'est pas d'avoir créé le Waterfall. L'erreur a été de croire qu'il était universel. Aucune approche n'est universelle. La sagesse, c'est de savoir quand et comment utiliser chaque outil dans la grande boîte à outils de la réalisation logicielle. »*
>
> - ŢŢĻŞŢ~ - Philosophie ORIGINALIUM

CHAPITRE 4

Agile et la réconciliation avec Totalisation

Le manifeste Agile : une révolution nécessaire

En février 2001, dix-sept développeurs de logiciels se sont réunis dans une station de ski à Snowbird, en Utah. Ils avaient des noms, des expériences et des idées différentes, mais ils partageaient tous un sentiment commun : quelque chose n'allait pas dans la façon dont le développement logiciel était pratiqué, et ce quelque chose avait un nom - les processus lourds, les spécifications exhaustives, la bureaucratie des phases séquentielles. De cette réunion est né le Manifeste Agile, l'un des documents les plus influents de l'histoire du développement logiciel.

Le Manifeste Agile proclamait quatre valeurs fondamentales. Les individus et leurs interactions au-delà des processus et des outils. Un logiciel fonctionnel au-delà d'une documentation exhaustive. La collaboration avec le client au-delà de la négociation contractuelle. La réponse au changement au-delà du suivi d'un plan. Ces quatre valeurs, dans le contexte de 2001, étaient révolutionnaires. Elles représentaient un changement de paradigme aussi fondamental que celui qu'avait opéré le Waterfall trente ans plus tôt. Et comme tous les changements de paradigme, elles contenaient à la fois une libération et un nouveau risque.

La libération était réelle et nécessaire. Elle libérait les équipes de la tyrannie des spécifications figées, des plans impossibles à respecter, des phases imperméables qui empêchaient l'apprentissage de se propager rapidement. Elle remettait les humains au centre du processus, affirmait que la collaboration valait mieux que la contractualisation, que l'adaptation valait mieux que la rigidité. C'était une révolution que le domaine du développement logiciel attendait depuis longtemps.

Ce qu'Agile a réparé - et ce qu'il a cassé en retour

Vingt-cinq ans après le Manifeste, il est possible - et nécessaire - de porter un regard honnête sur ce qu'Agile a réparé et sur ce qu'il a, dans certains de ses déploiements, cassé en retour. Ce regard n'est pas une critique du manifeste original. C'est un regard lucide sur la façon dont une vision philosophique se transforme, souvent maladroitement, en pratiques organisationnelles.

Ce qu'Agile a réparé avec succès, c'est avant tout la rigidité du cycle de vie. En introduisant les itérations courtes - les sprints de Scrum, les cycles de livraison continue - Agile a créé des moments de rétroaction réguliers qui permettent de corriger la trajectoire bien avant que les erreurs ne deviennent des catastrophes. Il a réparé aussi le divorce entre les équipes et leurs utilisateurs, en intégrant la collaboration continue avec le client comme une pratique centrale plutôt qu'une exception. Et il a réparé la culture de la documentation pour la documentation, en remettant l'accent sur la valeur livrée plutôt que sur les artefacts produits.

Mais dans son déploiement à grande échelle, dans sa transformation en industrie - consultants certifiés Scrum, formations en masse, cadres d'entreprise comme SAFe - Agile a aussi cassé certaines choses importantes. Il a souvent cassé la culture de la documentation, au point que beaucoup d'équipes "agiles" produisent aujourd'hui des systèmes dont l'architecture est incompréhensible pour

quiconque n'était pas là pendant le sprint où la décision a été prise. Il a parfois cassé la vision architecturale cohérente à long terme, en valorisant tellement la livraison rapide que la dette technique s'est accumulée sprint après sprint jusqu'à devenir un fardeau insoutenable. Et il a paradoxalement créé ses propres rituels rigides - les stand-ups quotidiens, les rétrospectives de sprint, les revues de sprint - qui, dans de nombreuses organisations, sont devenus aussi mécaniques et aussi vides de sens que les phases du Waterfall qu'ils étaient censés remplacer.

L'agilité mal comprise : vitesse sans qualité, itérations sans apprentissage

La corruption la plus profonde de l'Agile, celle que Totalisation diagnostique avec le plus de fermeté, c'est la réduction de l'agilité à la seule vitesse de livraison. Dans trop d'organisations, "être Agile" est devenu synonyme de "livrer vite" et de "changer de direction facilement". Ce n'est pas de l'agilité. C'est de la précipitation déguisée en vertu.

L'agilité authentique, telle que l'entendaient les signataires du manifeste original, implique la capacité à apprendre rapidement et à intégrer cet apprentissage dans la direction suivante. Le mot clé est "apprendre". Chaque itération devrait être une occasion d'apprentissage cumulatif : qu'est-ce qu'on a découvert sur le problème que l'on résout ? Qu'est-ce qu'on a appris sur nos utilisateurs ? Qu'est-ce que cette itération nous a révélé sur notre architecture, notre processus, notre collaboration ?

Quand les itérations sont des cycles de production purs - "finir les user stories du sprint, passer au sprint suivant" - sans réelle réflexion sur l'apprentissage qu'elles génèrent, alors elles ne sont pas agiles. Elles sont juste rapides. Et la rapidité sans apprentissage est l'une des façons les plus efficaces de produire un logiciel de mauvaise qualité à grande vitesse.

La distinction cruciale de Totalisation

Il y a une différence fondamentale entre une organisation qui *fait* de l'Agile et une organisation qui *est* agile. La première a adopté les cérémonies, les artefacts et les rôles. La seconde a intégré la posture d'apprentissage continu, d'adaptation consciente et de responsabilité collective qui constitue l'essence de l'agilité authentique. Totalisation s'intéresse à la seconde.

La vision ŢŢĻŞŢ~ sur l'Agile : l'itération comme outil, pas comme religion

Totalisation propose une vision de l'Agile qui le libère de son propre dogmatisme. Et oui, l'Agile a développé son propre dogmatisme - le comble pour un mouvement qui se définissait par opposition à la rigidité du Waterfall. Ce dogmatisme se manifeste dans la croyance que toute organisation qui fait du développement logiciel devrait adopter des itérations courtes, des rôles Scrum ou équivalents, et les quatre valeurs du manifeste comme des vérités absolues et universelles.

La vision ŢŢĻŞŢ~ est différente. Elle voit l'itération - le principe de diviser le travail en incréments livrables avec des points de rétroaction réguliers - comme un outil extraordinairement puissant. Mais un outil, pas une religion. Un outil qui est approprié dans de nombreux contextes, qui peut être adapté et modulé selon les besoins spécifiques de chaque projet, de chaque équipe, de chaque organisation. Un outil dont l'utilisation intelligente requiert du jugement, de la nuance et une compréhension profonde de ce qu'on cherche à accomplir.

De la même façon, Totalisation voit les valeurs du Manifeste Agile non pas comme des absolus, mais comme des pendules. "Les individus et leurs interactions *au-delà* des processus et des outils" ne signifie pas "ignore les processus et les

outils". Cela signifie que quand tu dois choisir entre un processus rigide et les besoins réels d'un individu ou d'une interaction humaine, privilégie l'humain. Mais si tu n'as pas à choisir - si un processus bien conçu soutient et amplifie les interactions humaines plutôt que de les contraindre - alors utilise ce processus avec joie.

Agile et Totalisation : une réconciliation productive

La réconciliation entre Agile et Totalisation est, de toutes les réconciliations explorées dans cet ouvrage, celle qui porte peut-être le plus immédiatement de valeur pratique. Parce que la grande majorité des organisations qui développent des logiciels aujourd'hui utilisent une forme ou une autre d'Agile, et beaucoup d'entre elles sont aux prises avec les limites et les dérives que nous avons décrites.

Cette réconciliation commence par un geste simple : ramener la conscience de la continuité dans des organisations qui se sont fragmentées en sprints. Un sprint est une unité de temps utile pour organiser le travail et créer des points de rétroaction. Mais il ne devrait jamais être une frontière mentale. Ce que tu construis dans ce sprint est la continuité de ce que tu as construit dans les sprints précédents et la fondation de ce que tu construiras dans les sprints futurs. Cette conscience de la continuité - avant, pendant et après - est précisément ce que Totalisation apporte à la pratique agile.

Concrètement, une équipe qui pratique l'Agile à travers le prisme de Totalisation ne se demande pas seulement "qu'est-ce qu'on va livrer dans ce sprint ?" Elle se demande aussi "comment ce qu'on va livrer dans ce sprint s'inscrit-il dans la vision globale du produit ?" Elle se demande "quelles décisions prenons-nous maintenant qui vont contraindre ou libérer notre capacité d'action dans les sprints

futurs ?" Elle se demande "quel apprentissage du sprint précédent intégrons-nous consciemment dans la façon dont nous abordons ce sprint ?"

Ces questions ne ralentissent pas l'équipe. Elles l'enrichissent. Elles transforment une collection de sprints en un véritable apprentissage cumulatif, en une progression cohérente vers un objectif partagé et compris par tous. Elles transforment l'Agile d'un outil de production rapide en une posture de réalisation totale - avant, pendant et après, en continu, sans interruption.

> *« L'Agile sans Totalisation est une série de présents sans passé et sans futur. Totalisation sans Agile est une vision sans ancrage dans l'action. Ensemble, ils constituent quelque chose de plus grand que chacun d'eux séparément : un cycle de vie vraiment vivant. »*
>
> - Léo Michel Bélanger, ŢŢĻŞŢ~ - Philosophie ORIGINALIUM

Cette réconciliation n'est pas la fin du chemin. C'est le début. Elle pose les fondations d'une compréhension renouvelée du SDLC - une compréhension dans laquelle Waterfall et Agile ne sont plus des adversaires idéologiques, mais des perspectives complémentaires que Totalisation intègre dans une vision plus large, plus riche et plus humaine de ce que signifie réaliser un logiciel.

Note de fin de Partie A - Aperçu de la Partie B

Tu arrives à la fin de la Partie A de ŢŢĻŞŢ~. Ce que tu viens de traverser n'est pas un résumé de méthodologies. C'est un changement de regard. Un changement de regard sur le SDLC lui-même, sur la fragmentation qui

l'a blessé, sur le Waterfall et sur l'Agile - deux approches que Totalisation refuse de condamner et choisit au contraire de réconcilier dans une vision plus profonde.

Tu comprends maintenant le diagnostic de Totalisation : le problème du SDLC n'est pas dans ses outils, ni dans ses méthodes. Il est dans la façon dont on conçoit le cycle de vie lui-même - comme une séquence de phases étanches plutôt que comme un flux continu et vivant. Et tu comprends la réponse que propose ŢŢĻṢŢ~ : la conscience de la continuité, la responsabilité de l'ensemble, l'action ancrée simultanément dans l'avant, le pendant et l'après.

La Partie B poursuivra ce voyage en explorant :

Le mouvement DevOps et sa réconciliation avec Totalisation - parce que DevOps représente une tentative réelle et précieuse de réconcilier le développement et les opérations, et Totalisation a beaucoup à dire sur ce qu'il a accompli et sur ce qu'il laisse encore en suspens.

Le modèle en spirale et les approches hybrides - ces tentatives d'allier la rigueur du Waterfall et la flexibilité de l'Agile, vues à travers le prisme totalisateur.

La dimension humaine du SDLC - parce que le cycle de vie du logiciel est avant tout un cycle de vie humain, et que Totalisation n'est pas une philosophie de processus mais une philosophie de personnes.

Et enfin, la vision synthétique : comment ŢŢĻṢŢ~ propose de réconcilier l'ensemble de ces approches dans une posture unifiée et totalisante - avant, pendant et après, en continu, sans interruption.

La Partie B arrive. Le flux continue.

ŢŢĻŞŢ~ - Réconciliation avec le cycle de vie de la réalisation du logiciel (SDLC) - Partie A

© Léo Michel Bélanger - Tous droits réservés | Images et couverture par Soléane Bélanger

Publié par ORIGINALIUM | ŢŢĻŞŢ~ - Montréal, Québec, Canada - 2026